CSCI-4208: Developing Advanced Web Applications

Week 9: Lecture 13

JS Modules (Browser)

Modules - Overview

- Modules: Introduction
- Modules: HTML
- Modules: JavaScript
- Modules: Exporting a Module
- Modules: Importing a Module
- Modules: Default Export
- Modules: Combining Exports: Default & Named
- Modules: Aggregating Modules

Modules - Introduction

As our application grows bigger, we want to split it into multiple files, so called “modules”. A module may contain a class or a library of functions for a specific purpose.

What is a module?

A module is just a file. One script is one module. As simple as that.

Modules can load each other and use special directives `export` and `import` to interchange functionality, call functions of one module from another one:

- `export` keyword labels variables and functions that should be accessible from outside the current module.
- `import` allows the import of functionality from other modules.

Modules - Introduction

What is a Module?

A module organizes a related set of JavaScript code. A module can contain variables and functions. A module is nothing more than a chunk of JavaScript code written in a file.

By default, variables and functions of a module are not available for use. Variables and functions within a module should be exported so that they can be accessed from within other files. Modules in ES6 work only in strict mode. This means variables or functions declared in a module will not be accessible globally.

Modules - Introduction

Why use modules?

There are many benefits to using modules in favor of a sprawling, interdependent codebase.
The most important ones, in my opinion, are:

1) Maintainability:

By definition, a module is self-contained. A well-designed module aims to lessen the dependencies on parts of the codebase as much as possible, so that it can grow and improve independently. Updating a single module is much easier when the module is decoupled from other pieces of code.

Modules - Introduction

Why use modules?

There are many benefits to using modules in favor of a sprawling, interdependent codebase.
The most important ones, in my opinion, are:

2) Namespacing:

In JavaScript, variables outside the scope of a top-level function are global (meaning, everyone can access them).
Because of this, it’s common to have “namespace pollution”, where completely unrelated code shares global variables.

Sharing global variables between unrelated code is poor software design. Modules allow us to avoid namespace pollution by creating a private space for our variables.

Modules - Introduction

Why use modules?

There are many benefits to using modules in favor of a sprawling, interdependent codebase.
The most important ones, in my opinion, are:

3) Reusability:

Let’s be honest here: we’ve all copied code we previously wrote into new projects at one point or another. For example, let’s imagine you copied some utility methods you wrote from a previous project to your current project.

That’s all well and good, but if you find a better way to write some part of that code you’d have to go back and remember to update it everywhere else you wrote it.

This is obviously a huge waste of time. Wouldn’t it be much easier if there was — a module that we can reuse over and over again?

Modules - Introduction

Difference between modules & standard scripts

- Cannot load modules locally (i.e. with a `file://` URL), you'll get CORS errors due to JavaScript module security requirements. You must to do any testing through a http://.

- Module features are imported into the scope of a single script — they aren't available in the global scope. Therefore, you will only be able to access imported features in the script they are imported into, and you won't be able to access them from the JavaScript console.

- `this` within modules does not refer to the global `this`, and instead is `undefined`

- The import and export syntax is only available within modules — it doesn’t work in classic scripts.

Modules - Introduction

Technical Info: Module loading

When the JS engine runs a module, it goes through four steps:

1. **Parsing:** The implementation reads the source code of the module and checks for syntax errors.

2. **Loading:** The implementation loads all imported modules (recursively).

3. **Linking:** For each newly loaded module, the implementation creates a module scope and fills it with all the bindings declared in that module, including things imported from other modules.

4. **Run time:** Finally, the implementation runs the statements in the body of each newly-loaded module.

Modules - HTML

Using JS modules in the browser

On the web, you can tell browsers to treat a <script> element as a module by setting the type attribute to module.

```
<script type="module" src="main.js"></script>
```

Modules - HTML

Modules only work with HTTP

JavaScript modules load multiple scripts more effectively using HTTP GET request to perform non-blocking asynchronous imports. However, for this reason modules only work with HTTP protocol and not file://.

- If you attempt to use file:// protocol to open an HTML that contains javascript modules, you will get a CORS error

Modules - JavaScript

Module Pattern

The Module pattern is used to mimic the concept of encapsulation across scripts so that we can store both public and private methods and variables inside a single script without concerns of a global scope. That allows us to create a public facing API for the methods that we want to expose to the world, while still encapsulating private variables and methods in a closure scope.

Modules - JavaScript

Module Specifiers

When importing modules, the string that specifies the location of the module is called the “module specifier” or the “import specifier”.

In example, the module specifier is './lib.js':

Modules - Exporting a Module

The export keyword can be used to export components in a module. Exports in a module can be classified as follows −

- Named Exports
- Default Exports

Modules - Exporting a Module

Named Exports

Named exports are distinguished by their names. There can be several named exports in a module. A module can export selected components using the syntax given below −

Syntax 1

```
//using multiple export keyword
export component1
export component2
...
...
 export componentN
```

Syntax 2

Alternatively, components in a module can also be exported using a single export keyword with {} binding syntax as below

```
//using single export keyword

 export {component1,component2,....,componentN}
```

Modules - Exporting a Module

Default Exports

Modules that need to export only a single value can use default exports. There can be only one default export per module.

Syntax

```
export default component_name
```

However, a module can have a default export and multiple named exports at the same time.

Modules - Importing a Module

To be able to consume a module, use the **import keyword**. A module can have multiple **import statements**.

Importing Named Exports

While importing named exports, the names of the corresponding components must match.

Syntax

```
import {component1,component2..componentN} from './module_name.js'
```

However, while importing named exports, they can be renamed using the **as** keyword. Use the syntax given below −

```
import {original_component_name as new_component_name } from './module_name.js'
```

All named exports can be imported onto a module object by using the asterisk * operator.

```
import * as variable_name from './module_name.js'
```

Modules - Importing a Module

Importing Default Exports

Unlike named exports, a default export can be imported with any name.

Syntax

```
import any_variable_name from './module_name.js'
```

Example: Named Exports

Step 1 − Create a file *company1.js* and add the following code −

```
let company = "Foobar"

let getCompany = function(){
   return company.toUpperCase()
}

let setCompany = function(newValue){
   company = newValue
}

 export {company,getCompany,setCompany}
```

Modules - Importing a Module

Step 2 − Create a file *company2.js*. This file consumes components defined in the *company1.js* file. Use any of the following approaches to import the module.

Approach 1

```
import {company,getCompany} from './company1.js'

console.log( company, getCompany() )
```

Approach 2

```
import {company as x, getCompany as y} from './company1.js'

console.log( x, y() )
```

Approach 3

```
import * as myCompany from './company1.js'

console.log( myCompany.getCompany(), myCompany.company  )
```

Modules - Importing a Module

Step 3 − Execute the modules using an HTML file

To execute both modules we need a HTML file as shown below and run this in HTTP. Note that we should use the attribute `type="module"` in the script tag.

```
<html>
<head>
   <title>Document</title>
</head>
<body>
   <script src="./company2.js" type="module"></script>
</body>
 </html>
```

The output of the above code will be as stated below −

```
Foobar
FOOBAR
```

Modules - Default Export

Step 1 − Create a file *company1.js* and add the following code −

```
let name = 'Foobar'

let company = {
   getName: () => name,
   setName: newName => name = newName
}

export default company
```

Step 2 − Create a file *company2.js*. This file consumes the components defined in the *company1.js* file.

```
import c from './company1.js'
console.log(c.getName())
c.setName('Barfoo')
console.log(c.getName())
```

Modules - Default Export

Step 3 − Execute the modules using an HTML file

To execute both the modules we need to make an html file as shown below and run this in live server. Note that we should use the attribute type="module" in the script tag.

```
<html>
<head>
   <title>Document</title>
</head>
<body>
   <script src="./company2.js" type="module"></script>
</body>
</html>
```

The output of the above code will be as stated below −

```
Foobar
Barfoo
```

Modules - Combining Exports: Default & Named

Step 1 − Create a file *company1.js* and add the following code −

```
//named export
export let name = 'Foobar'

let company = {
   getName: () => name,
   setName: (newName) => name = newName
}
//default export
 export default company
```

Step 2 − Create a file *company2.js*. This file consumes the components defined in the *company1.js* file. Import the default export first, followed by the named exports.

```
import c, {name} from './company1.js'

console.log(name)
console.log(c.getName())
c.setName("Barfoo")
console.log(c.getName())
```

Modules - Combining Exports: Default & Named

Step 3 − Execute the modules using an HTML file

```
<html>
   <head>
      <title>Document</title>
   </head>
   <body>
      <script src="company2.js" type="module"></script>
   </body>
</html>
```

The output of the above code will be as shown below −

```
Foobar
Foobar
Barfoo
```

Modules - Aggregating Modules

Some modules may just be responsible for aggregating the exports of other modules.

There's an all-in-one import-and-export shorthand:

```
// world-foods.js - good stuff from all over


export {Tea, Cinnamon} from "./sri-lanka.js";
export {Coffee, Cocoa} from "./equatorial-guinea.js";
export * from "./singapore.js";
```

Each one of these `export-from` statements is similar to an `import-from` statement followed by an `export`.

The End.

Coming Soon... to a Moodle near you!

Labs	Quiz Game with Leaderboard
Homework	REST Client App using Modules pattern

Modules - References

- https://hacks.mozilla.org/2015/08/es6-in-depth-modules/
-